Elliptic Curve Application

Yuxiang Sun

October 2025

1 Introduction

Modern cryptography builds on two main paradigms: symmetric encryption and asymmetric (public-key) encryption. Symmetric ciphers such as AES are extremely efficient and, when keys are kept secret, practically infeasible to break. However, they suffer from the key distribution problem: the same key is used to encrypt and decrypt, so Alice must somehow deliver that key to Bob over an insecure Internet. Any attempt to transmit the key alongside the ciphertext exposes it to interception by adversaries, defeating confidentiality. Public-key cryptography addresses this limitation. Bob publishes a public key that anyone—including Alice—can use to encrypt messages, while only Bob's private key can decrypt them. This removes the need to share a decryption key over an untrusted channel. In practice, systems often use a hybrid approach: Alice generates a random symmetric session key, encrypts it with Bob's public key, and then uses AES with that session key to protect the bulk data, combining the performance of symmetric encryption with the safe key establishment provided by asymmetric methods.

RSA and elliptic-curve cryptography are the two best-known public-key algorithms. Compared with the traditional RSA scheme, the primary advantage of elliptic-curve cryptography (ECC) is that it can provide the same level of security with much shorter key lengths. This translates into faster computation, lower energy consumption, and reduced storage requirements, making ECC particularly well-suited for modern applications.

2 What is Elliptic-Curve Cryptography?

Elliptic-curve cryptography, as its name suggests, is based on operations defined over elliptic curves. This naturally raises the question of what specific form elliptic curves take in cryptographic applications, which operations are employed to enable encryption, and what additional constraints must be imposed on the curves to ensure security and correctness.

Unlike traditional algorithms such as RSA, which rely purely on number theory, the basic elements of elliptic-curve cryptography are points in a two-dimensional Cartesian coordinate system. Formally, the set of all elements of an elliptic curve over a finite field can be expressed as

$$\mathbb{S} = \{(x, y) \in (\mathbb{F}_{\text{ı}})^2 \mid y^2 = x^3 + ax + b \pmod p, 4a^3 + 27b^2 \neq 0 \pmod p\} \cup \{\mathcal{O}\}$$

where $\mathcal{O}$ denotes the point at infinity serving as the group's identity element.

Actually, infinity is a very tricky terminology since it is a little bit counterintuitive. In different situation, the infinity has slight difference (maybe it is because of my limited knowledge). I will explain it in detail later.

Unfortunately, I have already used the term group. My original intention was to introduce the notion later, emphasizing that elliptic-curve cryptography relies on group properties to enable its operations. However, since it is necessary to explain the element $\{\mathcal{O}\}$—the point at infinity—I was compelled to invoke the concept of a group at this stage. Having mentioned it, we will now provide a detailed introduction to the elliptic curve as a group.

3 Group Behind Elliptic-Curve Cryptography

To begin with, it should be noted that the elegant properties and operations of elliptic-curve cryptography are fundamentally built upon the structure of an Abelian group. Now, let's construct this Abelian group together.

A group needs two components: set and operation. Here, $\mathbb{S}$ is our set. In terms of operation, just think about it, what operation can be applied to 2 points and interact with whole elliptic curve? Draw a line through these two points! So, after this operation, what is the result? Remember, in this group, the element is points not numbers. Therefore, the result should be a point. Naturally, the intersection of the line and elliptic curve. But how many intersection points it will generate? Here is the theorem.

Theorem 1. *If F is a field and $P(x) \subset F[x]$ is a nonzero polynomial of degree n, then the set of distinct roots of $P(x)$ in F contains at most n elements. Equivalently, the number of roots of $P(x)$ in F, counted with multiplicity, is exactly n.*

Returning to our problem, a line and an elliptic curve can intersect in at most three points. This means that if you start with two points P and Q, the geometry guarantees at most one additional intersection point. But how do we ensure that there is exactly one more point? This is where the point at infinity comes into play. We just assume another intersection is point at infinity. It might seem that there are infinitely many such points at infinity, since P and Q are chosen arbitrarily and each (P, Q) pair can generate a different new point. But what happens if P is the point at infinity, or if Q is the point at infinity? Without loss of generality, let us consider two cases:

- P is point at infinity

  In this situation, we typically assume that the line through P and Q is vertical to the x-axis. At first, this seems counterintuitive: if there are infinitely many points at infinity and no inherent restriction on which one P represents, why should we take the line to be vertical? The assumption feels artificial, yet it leads to a consistent and workable definition.

- Both P and Q are point at infinity

  The construction becomes even more peculiar: by convention, the point they generate is taken to be at infinity. If the line intersects the elliptic curve elsewhere, why should the resulting point not lie on the curve? Nonetheless, the rule simply declares the outcome to be the point at infinity. As before, the assumption may feel unnatural, but it ensures that the group law remains coherent and mathematically consistent.

We have discussed the situation where exists at least one point at infinity. But what happens all three points are on the elliptic curve? Before giving the group law of elliptic curve, I give the definition of group first.

Definition 1.

(1) A group is an ordered pair $(G, \star)$ where G is a set and $\star$ is a binary operation on G satisfying the following axioms:

(i) $(a \star b) \star c = a \star (b \star c)$, for all $a, b, c \in G$, i.e., $\star$ is associative,

(ii) there exists an element e in G, called an identity of G, such that for all $a \in G$ we have $a \star e = e \star a = a$,

(iii) for each $a \in G$ there is an element a^{-1} of G, called an inverse of a, such that $a \star a^{-1} = a^{-1} \star a = e$.

(2) The group $(G, \star)$ is called abelian (or commutative) if $a \star b = b \star a$ for all $a, b \in G$.

Now let me give the systematic explanation of how elliptic curves are formally defined.

- **Element:** the elements of the group are the points of an elliptic curve;
- **Identity:** the **identity element** is the point at infinity $\mathcal{O}$;
- **Inverse:** the **inverse** of a point P is the one symmetric about the x-axis;
- **Operation: addition** is given by the following rule: if three non-zero points P, Q, and R are collinear, then their sum is $P + Q + R = \mathcal{O}$.

Remark: By the **addition rule**, we will find $P + Q = -R$. Because $-R$ represents the inverse of R, we know the result of addition is defined as the reflection (inverse) of the third intersection point between the line through P and Q and the elliptic curve.

I can give some graphs to show how it looks like. Figure 1 is when P, Q are on the curve and $P \neq Q$ and the third intersection exists. Figure 2 is when P, Q are on the curve and $P = Q$ and the third intersection exists. Figure 3 is when P is $\mathcal{O}$. Figure 4 is when P, Q are on the curve, but another point $R = \mathcal{O}$.

In many articles or blog posts on elliptic curves, it is often stated directly that the set of points on an elliptic curve forms a group. But why is this the case? To be rigorous, we need to prove it. The proof is given as follows:

1. Closure:

   Case 1: If the line through P and Q has the third intersection with the curve, then we pick its inverse.

   Case 2: If the line through P and Q does not have another intersection with the curve, then the third point is $\mathcal{O}$

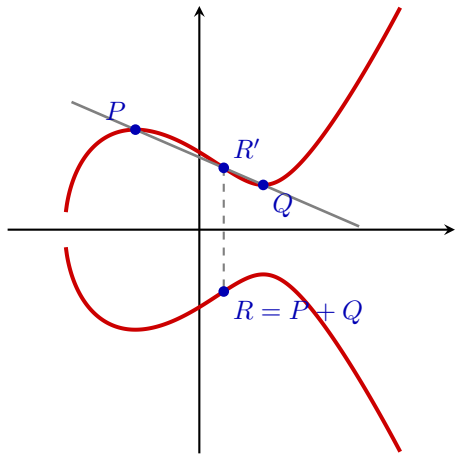

Figure 1: Case $P \neq Q$

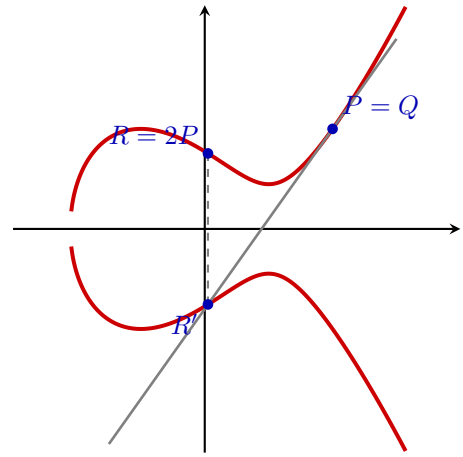

Figure 2: Case $P = Q$ (tangent line)

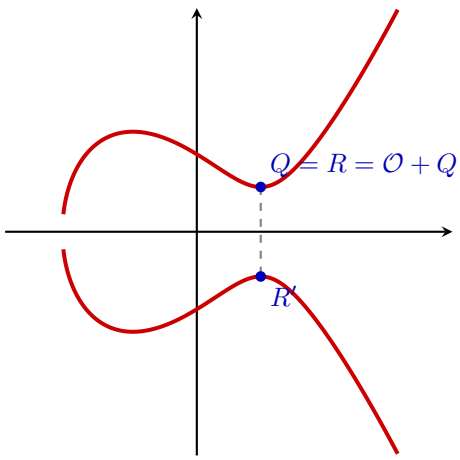

Figure 3: Case $P = \mathcal{O}$ (identity)

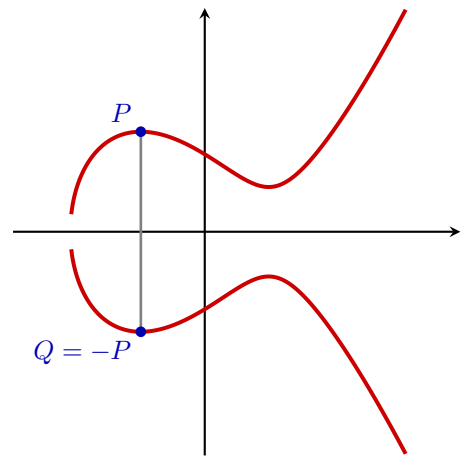

Figure 4: Case $P + Q = \mathcal{O}$ (inverses)

2. Identity:

   Obviously, $\mathcal{O}$ is the identity

3. Inverse:

   The elliptic curve is symmetric to the x-axis, thus for any point $R = (x, y)$, its inverse is $(x, -y)$. If $R = \mathcal{O}$, then its inverse is itself $\mathcal{O}$.

4. Associativity:

   This is the most tricky one. Actually, there are many different ways to prove it. The most straightforward way is using algebraic calculation, but it is very complicated. Here, I choose a more elegant way to prove this.

Okay, these group properties are fancy but how to relate these rules to cryptography? How to use elliptic curve to do something useful? The typical application is Elliptic Curve Diffie-hellman Key Exchange.

4 Elliptic Curve Diffie-Hellman Key Exchange

Elliptic-Curve Diffie–Hellman (ECDH) is essentially the elliptic-curve–based solution to the key distribution problem discussed in the introduction. By leveraging asymmetric cryptography, it enables two parties to securely agree on and share a secret key, which can then be used for symmetric encryption.

The security of ECDH is based on the computational difficulty of the Elliptic Curve Discrete Logarithm Problem (ECDLP), which makes it infeasible for a passive eavesdropper to determine the shared secret from the publicly

exchanged information. ECDH offers equivalent security to traditional Diffie-Hellman with significantly smaller key sizes, leading to improved performance and reduced computational overhead.

Next, I will outline the specific steps of the Elliptic-Curve Diffie–Hellman (ECDH) protocol. Suppose Alice and Bob wish to exchange messages securely, while an eavesdropper, Eve, attempts to intercept their communication.

4.1 The procedure of ECDH

Step 0: Domain Parameter Agreement Before the key exchange can begin, Alice and Bob must agree on a set of public **Elliptic Curve Domain Parameters**. These parameters define the mathematical context for the operations. The domain parameters consist of:

- **An Elliptic Curve Equation**: Alice and Bob must agree on the form of elliptic curve over a finite field.
- **A Finite Field**: Choose the operation is defined on which field, usually people just choose $\mathbb{F}_{|}$
- **A Base Point G**: A publicly known point on the curve that serves as a generator for the subgroup of points.
- **The Order n of the Base Point**: A large prime number representing the number of points in the subgroup generated by G.

With so many parameters involved, one might wonder how Alice and Bob can efficiently select them, especially since the numbers n and p are extremely large. In practice, they usually rely on well-established protocols that provide standardized public parameters, so they do not need to generate them on their own. However, if Alice and Bob do not trust any existing standards and wish to construct their own, the process works as follows. First, they agree on a large prime number n. Next, they choose an elliptic curve and a prime p such that the number of points on the curve satisfies $|S| = N = kn$, where S denotes the set of points on the curve and $k \in \mathbb{N}$. Then, they randomly select a point P on the curve and compute their base point as $G = \frac{N}{n} \cdot P$. If the computed G does not lie on the curve, they repeat the process with a different P until a valid G is obtained. The resulting point G is then chosen as the base point, and it can be shown that the order of G is exactly n. Before presenting the proof of this fact, let us first introduce two theorems that will be used in the proof.

Theorem 2 (Lagrange's Theorem)**.** *Let G be a finite group and let $H \leq G$ be a subgroup of G. Then the order (number of elements) of H divides the order of G. Formally,*

$$|G| = [G : H] \cdot |H|,$$

where $[G : H]$ is the index of H in G.

Theorem 3 (Order Divides Exponent)**.** *Let G be a group and let $g \in G$ be an element of finite order $k = |g|$. If $g^s = e$ for some integer s, then $k \mid s$.*

Then we can use these 2 theorems to prove our conclusion. Proof is as follows:

Proof.

$$n \cdot G = n \cdot \frac{N}{n} \cdot P = N \cdot P = e$$

Therefore, we know

$$|G| \mid n$$

However, n is a prime number $\Rightarrow |G| = n$ □

After step 0, they have decided all parameters (p, a, b, G, n, h), where $h = \frac{N}{n}$ and other parameters have been explained above.

Step 1: Key Pair Generation Alice and Bob generate their own private and public keys independently.

- **Private Key**:
  - Alice selects a random integer d_A in the range $[1, n-1]$. This integer is her **private key** and must be kept secret.
  - Bob similarly selects a random integer d_B in the same range to be his **private key**.
- **Public Key**:
  - Alice computes her **public key**, Q_A, by performing scalar multiplication of the base point G with her private key d_A:
  $$Q_A = d_A \times G$$
  - Bob computes his **public key**, Q_B, in the same manner:
  $$Q_B = d_B \times G$$

Scalar multiplication $(d \times G)$ is the process of adding the point G to itself d times.

Step 2: Public Key Exchange

Alice and Bob exchange their public keys over the insecure channel.

- Alice sends her public key Q_A to Bob.
- Bob sends his public key Q_B to Alice.

An eavesdropper, Eve, can intercept Q_A and Q_B. However, due to the ECDLP, Eve cannot feasibly compute the private keys d_A or d_B from the public keys and the public domain parameters.

Step 3: Shared Secret Calculation

Alice and Bob can now independently compute the same shared secret using their own private key and the public key they received from the other party.

- **Alice's Calculation**: Alice uses her private key d_A and Bob's public key Q_B to compute the shared secret point S:
$$S = d_A \times Q_B = d_A \times (d_B \times G)$$
- **Bob's Calculation**: Bob uses his private key d_B and Alice's public key Q_A to compute the shared secret point S:
$$S = d_B \times Q_A = d_B \times (d_A \times G)$$

Because scalar multiplication on elliptic curves is associative, both parties arrive at the exact same point S:

$$d_A \times (d_B \times G) = d_B \times (d_A \times G)$$

The calculated point S is the shared secret. Typically, the x-coordinate of the point S is taken and passed through a key derivation function (KDF) to produce the final symmetric key for encryption. Now, we have solved the key distribution problem and safely generate a shared key by Alice and Bob. Then Alice and Bob can use this key to encrypt their message.

4.2 Illustrative Example

To demonstrate the process, consider a simplified example with small numbers.

- **Domain Parameters**:
  - Curve Equation: $y^2 = x^3 + 7$ over the finite field F_{23}.
  - Base Point $G = (2, 5)$.
- **Key Generation**:
  - Alice chooses private key $d_A = 4$.
  - Alice computes her public key: $Q_A = 4 \times (2, 5) = (17, 20)$.
  - Bob chooses private key $d_B = 7$.
  - Bob computes his public key: $Q_B = 7 \times (2, 5) = (11, 5)$.
- **Public Key Exchange**:
  - Alice sends $Q_A = (17, 20)$ to Bob.
  - Bob sends $Q_B = (11, 5)$ to Alice.
- **Shared Secret Calculation**:
  - Alice computes: $S = d_A \times Q_B = 4 \times (11, 5) = (8, 12)$.
  - Bob computes: $S = d_B \times Q_A = 7 \times (17, 20) = (8, 12)$.

Both parties successfully derive the same secret point $(8, 12)$. They can now use its x-coordinate, 8, as the shared secret for generating an encryption key.

4.3 Security Principle: The Elliptic Curve Discrete Logarithm Problem (ECDLP)

The security of the ECDH protocol is entirely dependent on the difficulty of the ECDLP. The problem can be stated as follows:

Given the public domain parameters (curve, field, G), and a public key point Q which is a multiple of G (i.e., $Q = dG$), it is computationally infeasible to find the integer d.

In the context of our example, an attacker may know G, Q_A, and Q_B, but they cannot efficiently determine d_A or d_B. Without a private key, the attacker cannot compute the shared secret point S. This property ensures that the key exchange is secure against passive eavesdroppers.

5 Elliptic Curve Digital Signature Algorithm

Digital signature is very important in many applications. When we download software from official websites, it typically comes with a digital signature. We need to verify the digital signature first to ensure the authenticity of the source and that the content has not been tampered with. When you log into a remote server, digital signatures are used to verify both your identity and the server's identity. When we access websites starting with https://, the SSL/TLS certificate presented by the server uses digital signatures to verify the authenticity of the certificate. Given their extensive use in practice, what are the fundamental principles behind digital signatures that make them trustworthy?

5.1 Create Signature

Let's say Alice wants to send a message to Bob. In order to ensure the message Bob receives is indeed what she sent, Alice creates a digital signature on the message.

Main Parameters:

1. k: private key
2. G: base point on elliptic curve
3. $K = k \cdot G$: public key
4. m: message
5. H: hash function
6. $h = H(m)$: using hash function to get a hashed value based on message m

Compute Signature (r, s):

1. generate a new, private random number z
2. compute r

   First Alice computes a temporary point $P = z \cdot G$, then she take the x-coordinate of P as r which means $r = P_x$
3. compute s

   Using the formula:
   $$s = z^{-1}(h + k \cdot r) \ (mod\ n)$$
   where n is the order of G

Send message m with signature (r, s):

Now Alice has computed her signature $(r, s) = (P_x, z^{-1}(h + k \cdot r) \ (mod\ n))$. She sends the message m signed by (r, s) and then deletes the random number z. Since z should be kept private and each time we need a new generated random number z with no duplication.

5.2 Verify Signature

Bob receives the message m and the signature (r, s). He wants to verify the authentication of the signature. Specifically, he wants to verify if r is the correct value. First, he computes the hash value $h = H(m)$. Then according to the equation $s = z^{-1}(h + k \cdot r) \ (mod\ n)$, he can get $z = s^{-1}(h + k \cdot r)$. Multiply G on both sides, we can get $z \cdot G = (s^{-1}(h + k \cdot r) \cdot G) = s^{-1}hG + s^{-1}krG = (s^{-1}h)G + (s^{-1}r)(kG)$. Because $P = zG$ and $K = kG$, then we have $P = (s^{-1}h)G + (s^{-1}r)K$. Since we have already known the value of s, h, G, K, r, then we can compute the x-coordinate of P. If $P_x = r$, then it is Alice's signature, otherwise is not.

5.3 Limitations

It seems everything is perfect so far. However, it still has many limitations.

1. quantum computing attack

   The security of traditional Elliptic Curve Cryptography (ECC) relies on the difficulty of the Elliptic Curve Discrete Logarithm Problem (ECDLP). While this problem is intractable for classical computers, a sufficiently powerful quantum computer running Shor's algorithm could solve it efficiently, thus breaking the cryptography.
2. critical dependence on random numbers

   The most classic example is the 2010 Sony PlayStation 3 ECDSA failure. The console used ECDSA to verify all software. However, Sony's engineers failed to generate a unique random number (k) for each signature, instead using the same static value every time. Attackers exploited this by comparing two different signatures, which allowed them to algebraically solve for this static k. Once k was known, they easily recovered Sony's master private key, enabling them to sign and run any custom software on the console.

3. lack of aggregation capability

   Imagine a blockchain block that needs to be approved by 1000 validators. If we use ECDSA, each of the 1,000 validators must produce their own individual signature. To validate the block, you must:

   (a) Store all 1000 signatures.

   (b) Verify all 1,000 signatures one by one.

   This is extremely costly in terms of storage space (data bloat) and computation (slow verification). The signature data for a block can become larger than the transaction data itself.

In this project, I will set aside the topic of quantum computing. I will instead focus on the BLS (Boneh-Lynn-Shacham) signature scheme. The primary goal is to analyze its powerful capability for signature aggregation, a feature critically employed by Ethereum's consensus layer.

6 Boneh-Lynn-Shacham Signature

The BLS signature scheme (designed by Boneh, Lynn, and Shacham) uses this pairing function to create a uniquely elegant and aggregatable signature. Before we start talking about BLS signature schema, I need to first introduce Bilinear Pairing.

6.1 Bilinear Pairing

Formally, a pairing e is a function that takes two inputs from two elliptic curve groups, G_1 and G_2, and outputs a single element in a "target" group, G_T.

$$e : G_1 \times G_2 \to G_T$$

This function has three essential properties:

1. **Bilinearity (The Key Property):** This is what makes everything possible. For any points $P \in G_1$, $Q \in G_2$, and any two secret numbers (scalars) a and b:

   $$e(a \cdot P, b \cdot Q) = e(P, Q)^{ab}$$

   This equation shows that scalars a and b (which could be private keys) can be "multiplied" together within the exponent, *even though you only know the public points* aP and bQ.

2. **Non-degeneracy:** The pairing isn't trivial. It doesn't just map everything to the same value. There must be points P and Q such that $e(P, Q) \neq 1$ (where 1 is the identity element of G_T).

3. **Computability:** The function e must be efficient to compute.

The security of pairing-based cryptography comes from the fact that while computing e is easy, the underlying discrete logarithm problem (finding a from aP) remains extremely difficult in all three groups (G_1, G_2, and G_T). Therefore, it still can not defend quantum computing attack.

6.2 The BLS Signature Principle

Setup First, all participants must agree on the public parameters:

- The three groups: G_1, G_2, and G_T.
- The bilinear pairing function: e.
- A generator for G_2: a public point $G \in G_2$.
- A public hash function H that maps any message m to a point in G_1. So, $H(m) \to Q_m \in G_1$.

(Note: The choice of which group to use for keys vs. signatures can be swapped, but we'll use a common one: public keys in G_2, signatures in G_1.)

Step 1: Key Generation (KeyGen)

- **Private Key (sk):** A signer (e.g., Alice) chooses a secret random number, sk.
- **Public Key (PK):** Alice computes her public key by multiplying the generator G by her private key:

$$PK = sk \cdot G$$

This PK is a point in G_2. Alice publishes PK for the world to see and keeps sk completely secret.

Step 2: Signing (Sign)

To sign a message m, Alice uses her **private key** sk:

1. **Hash the message:** Alice computes $Q_m = H(m)$, which is a point in G_1.
2. **Create the signature:** She multiplies this message-point Q_m by her private key sk.

$$\sigma = sk \cdot Q_m$$

The resulting signature, σ, is a single point in G_1.

Step 3: Verification (Verify)

A verifier (e.g., Bob) wants to check if Alice's signature σ is valid for the message m, using only her **public key** PK.

1. **Hash the message:** Bob computes the same message-point: $Q_m = H(m)$.
2. **Check the Pairing Equation:** Bob checks if the following equation is true. This is the core of BLS:

$$e(\sigma, G) \stackrel{?}{=} e(Q_m, PK)$$

Why does this work? Let's analyze both sides of the equation using the bilinearity property.

- **Left-Hand Side (LHS):**
  - We know $\sigma = sk \cdot Q_m$ (from Step 2).
  - So, $LHS = e(sk \cdot Q_m, G)$
  - By the bilinearity rule ($a = sk$, $b = 1$): $LHS = e(Q_m, G)^{sk}$
- **Right-Hand Side (RHS):**
  - We know $PK = sk \cdot G$ (from Step 1).
  - So, $RHS = e(Q_m, sk \cdot G)$
  - By the bilinearity rule ($a = 1$, $b = sk$): $RHS = e(Q_m, G)^{sk}$

Conclusion: Since $LHS = e(Q_m, G)^{sk}$ and $RHS = e(Q_m, G)^{sk}$, the equation $LHS = RHS$ holds true.

6.3 Concrete Example: Signature Aggregation

This is the feature used by Ethereum's consensus layer. Let's show how 3 validators (Alice, Bob, and Carol) can sign the *same message* m (e.g., "Block #12345 is valid") and have their signatures aggregated.

Individual Keys and Signatures

- **Alice:** $sk_A \to PK_A = sk_A \cdot G$
- **Bob:** $sk_B \to PK_B = sk_B \cdot G$
- **Carol:** $sk_C \to PK_C = sk_C \cdot G$

All three validators compute the same message-point: $Q_m = H(m)$. They each create their own signature:

- Alice signs: $\sigma_A = sk_A \cdot Q_m$
- Bob signs: $\sigma_B = sk_B \cdot Q_m$
- Carol signs: $\sigma_C = sk_C \cdot Q_m$

The Inefficient (Traditional) Way

Without aggregation, a verifier would have to check all three signatures separately. This requires **6 pairing operations**:

1. Check $e(\sigma_A, G) = e(Q_m, PK_A)$
2. Check $e(\sigma_B, G) = e(Q_m, PK_B)$
3. Check $e(\sigma_C, G) = e(Q_m, PK_C)$

For 10,000 validators, this would be **20,000 pairing operations**, which is computationally impossible.

The BLS Aggregation Way

An "aggregator" node (which can be anyone) simply collects all the individual signatures and public keys and **adds them together** (using elliptic curve point addition).

- **Aggregate Signature (σ_{agg}):**
$$\sigma_{agg} = \sigma_A + \sigma_B + \sigma_C$$
- **Aggregate Public Key (PK_{agg}):**
$$PK_{agg} = PK_A + PK_B + PK_C$$

The Single, Aggregated Verification

Now, a verifier only needs to perform **one** check using the aggregated data. This requires only **2 pairing operations**.

Verification Equation: $e(\sigma_{agg}, G) \stackrel{?}{=} e(Q_m, PK_{agg})$

Let's prove this works:

- **Left-Hand Side (LHS):**
$$\begin{aligned} LHS &= e(\sigma_A + \sigma_B + \sigma_C, G) \\ &= e((sk_A \cdot Q_m) + (sk_B \cdot Q_m) + (sk_C \cdot Q_m), G) \\ &= e((sk_A + sk_B + sk_C) \cdot Q_m, G) \\ &= e(Q_m, G)^{(sk_A + sk_B + sk_C)} \end{aligned}$$
- **Right-Hand Side (RHS):**
$$\begin{aligned} RHS &= e(Q_m, PK_A + PK_B + PK_C) \\ &= e(Q_m, (sk_A \cdot G) + (sk_B \cdot G) + (sk_C \cdot G)) \\ &= e(Q_m, (sk_A + sk_B + sk_C) \cdot G) \\ &= e(Q_m, G)^{(sk_A + sk_B + sk_C)} \end{aligned}$$

Conclusion: $LHS = RHS$. The single, aggregated signature is successfully verified.

This means that for 10,000 validators, the network only needs to store **one** small signature and perform **one** fast verification (with 2 pairings) to validate the entire set, making the consensus mechanism incredibly efficient.

7 Appendix

7.1 Proof of Elliptic Curve is a group

7.1.1 Definitions and Foundations

Definition 2 (Divisor Group)**.** *The group of divisors on E, denoted* $\mathrm{Div}(E)$*, is the free abelian group generated by the points of E. A divisor D is a formal finite sum of points:*

$$D = \sum_{P \in E(K)} n_P [P]$$

where $n_P \in \mathbb{Z}$ and all but finitely many n_P are zero. The group operation is formal addition.

Definition 3 (Degree of a Divisor)**.** *The degree of a divisor D is the sum of its coefficients:*

$$\deg(D) = \sum_{P \in E(K)} n_P$$

The map $\deg : \mathrm{Div}(E) \to \mathbb{Z}$ *is a group homomorphism.*

Definition 4 (Divisors of Degree Zero)**.** *This is the kernel of the degree map. It is the subgroup of* $\mathrm{Div}(E)$ *containing all divisors with degree 0:*

$$\mathrm{Div}^0(E) = \{D \in \mathrm{Div}(E) \mid \deg(D) = 0\}$$

As a subgroup of an abelian group, $\mathrm{Div}^0(E)$ *is itself an abelian group and is therefore associative.*

Definition 5 (Rational Functions and Principal Divisors)**.** *Let $K(E)$ be the field of rational functions on E. For any non-zero $f \in K(E)$, we can define its associated* ***principal divisor****,* $\mathrm{div}(f)$*, by summing its zeros and poles, weighted by their multiplicities:*

$$\mathrm{div}(f) = \sum_{P \in E(K)} \mathrm{ord}_P(f)[P]$$

A fundamental theorem states that any principal divisor on a projective curve has degree zero. Thus, the set of all principal divisors, $\mathrm{Prin}(E)$*, is a subgroup of* $\mathrm{Div}^0(E)$*.*

Definition 6 (The Picard Group)**.** *The (degree zero) Picard group of E is the quotient group:*

$$\mathrm{Pic}^0(E) = \mathrm{Div}^0(E)/\,\mathrm{Prin}(E)$$

This group consists of equivalence classes of degree-zero divisors, where two divisors D_1, D_2 are equivalent if their difference is a principal divisor ($D_1 - D_2 \in \mathrm{Prin}(E)$).

Remark. **Crucial Property:** Since $\mathrm{Div}^0(E)$ is an abelian group, its quotient $\mathrm{Pic}^0(E)$ is also an abelian group. **Its group operation is, by definition, associative.**

7.1.2 The Group Isomorphism

The core of this proof lies in asserting a fundamental theorem of elliptic curves, which is a consequence of the Riemann-Roch theorem. This theorem states that the group $(E, \oplus)$ is isomorphic to $(\mathrm{Pic}^0(E), +)$.

We define the map Ψ as follows:

Definition 7 (The Isomorphism Map)**.**

$$\Psi : E(K) \to \mathrm{Pic}^0(E)$$
$$\Psi(P) = [P] - [\mathcal{O}]$$

where $[D]$ denotes the equivalence class of the divisor D in $\mathrm{Pic}^0(E)$*. This map is well-defined, as* $\deg([P] - [\mathcal{O}]) = 1 - 1 = 0$*.*

We now state the two theorems upon which this proof rests.

Theorem 4 (Bijection)**.** *The map $\Psi : E(K) \to \mathrm{Pic}^0(E)$ is a bijection of sets. This means for every divisor class $C \in \mathrm{Pic}^0(E)$, there exists one and only one point $P \in E(K)$ such that $C = [P] - [\mathcal{O}]$.*

Theorem 5 (Homomorphism)**.** *The map Ψ is a group homomorphism. That is, for any $P, Q \in E(K)$:*

$$\Psi(P \oplus Q) = \Psi(P) + \Psi(Q)$$

Remark. The geometric group law $\oplus$ is specifically defined to make Theorem 5 true. The fact that $[P]+[Q]+[R]-3[\mathcal{O}]$ is a principal divisor (if P, Q, R are collinear) and the subsequent logic involving $[P] + [Q] - [P \oplus Q] - [\mathcal{O}]$ being principal is precisely the proof of this homomorphism property.

7.1.3 Proof of Associativity

We now use the established isomorphism Ψ to prove associativity.

Proof. Let P, Q, R be arbitrary points in $E(K)$. We wish to prove $(P \oplus Q) \oplus R = P \oplus (Q \oplus R)$.

Since Ψ is a bijection (Theorem 4), it is sufficient to prove that the images of the left-hand side (LHS) and right-hand side (RHS) under Ψ are equal.

Step 1: Compute the image of the LHS.

$$\begin{aligned} \Psi((P \oplus Q) \oplus R) &= \Psi(P \oplus Q) + \Psi(R) && \text{(by Theorem 5)} \\ &= (\Psi(P) + \Psi(Q)) + \Psi(R) && \text{(applying Theorem 5 again)} \end{aligned}$$

Step 2: Compute the image of the RHS.

$$\begin{aligned} \Psi(P \oplus (Q \oplus R)) &= \Psi(P) + \Psi(Q \oplus R) && \text{(by Theorem 5)} \\ &= \Psi(P) + (\Psi(Q) + \Psi(R)) && \text{(applying Theorem 5 again)} \end{aligned}$$

Step 3: Compare the results.

We are left with comparing two elements in $\mathrm{Pic}^0(E)$:

- LHS Image: $(\Psi(P) + \Psi(Q)) + \Psi(R)$
- RHS Image: $\Psi(P) + (\Psi(Q) + \Psi(R))$

As established in Section 2, $\mathrm{Pic}^0(E)$ is a quotient of the abelian group $\mathrm{Div}^0(E)$. The group law in $\mathrm{Pic}^0(E)$ (inherited from $\mathrm{Div}^0(E)$) is **associative by definition**.

Therefore:

$$(\Psi(P) + \Psi(Q)) + \Psi(R) = \Psi(P) + (\Psi(Q) + \Psi(R))$$

Step 4: Conclusion.

Since $\Psi(\text{LHS}) = \Psi(\text{RHS})$ and the map Ψ is a bijection (injective), the original elements must be equal:

$$(P \oplus Q) \oplus R = P \oplus (Q \oplus R)$$

This completes the proof. □